



Département des Technologie de
l'information et de la communication (TIC)
Informatique et systèmes de communication
Sécurité informatique

Travail de Bachelor

Rapport Intermédiaire

Portage de la pile de communication Nanostack (Wi-SUN) sur
Zephyr RTOS

Étudiant	Benoit Delay
Enseignant responsable	Prof. Favrat Pierre
Année académique	2025-26

Yverdon-les-Bains, le 29.03.2026

Table des matières

1	Glossaire des termes techniques	6
2	Introduction	9
2.1	Structure du document	10
3	Cahier des charges	11
3.1	Résumé du contexte	11
3.1.1	Problématique	11
3.2	Phase du projet	12
3.2.1	Livrables	13
4	Planification	15
4.1	Planification initiale	15
4.2	Diagramme de gantt initiale	17
5	État de l'art	19
5.1	Standard Wi-SUN	19
5.2	Mbed OS	19
5.3	Zephyr	20
5.3.1	Kernel Zephyr	20
5.3.2	Gestion de l'énergie	20
5.3.3	Connectivité et protocole	20
5.4	Nanostack	20
5.4.1	SAL (Socket Abstraction Layer)	21
5.4.2	HAL (Hardware Abstraction Layer)	21
	Bibliographie	23

TABLE DES MATIÈRES

1 Glossaire des termes techniques

Acronyme / Terme	Définition
RTOS	Real-Time Operating System : Système d'exploitation déterministe conçu pour traiter un événement et fournir une réponse dans un délai strict et garanti.
SAL	Software Abstraction Layer : Couche logicielle de la Nanostack qui offre une interface standardisée (API Socket) et gère les protocoles réseau indépendamment du matériel.
HAL	Hardware Abstraction Layer : Couche d'abstraction matérielle. Elle fait le lien entre le logiciel (comme la SAL) et les composants physiques (antenne radio, timers, etc.).
6LoWPAN	IPv6 over Low-Power Wireless Personal Area Networks : Protocole permettant de compresser et transmettre des paquets IPv6 sur des réseaux sans fil à faible consommation (comme le 802.15.4).
RPL	Routing Protocol for Low-Power and Lossy Networks : Protocole de routage dynamique utilisé par la Nanostack pour trouver le meilleur chemin dans un réseau maillé (mesh).
Mbed TLS	Bibliothèque de cryptographie autonome (anciennement liée à Mbed OS) fournissant les mécanismes de sécurité comme TLS et DTLS pour chiffrer les communications.
TRNG	True Random Number Generator : Générateur de nombres aléatoires matériel, indispensable à Mbed TLS pour créer des clés de chiffrement robustes.

Tableau 1. – Glossaire des termes techniques

2 Introduction

L'émergence des Smart Cities transforme radicalement la gestion des infrastructures urbaines en imposant une connectivité omniprésente et en temps réel. Ce changement d'échelle crée un besoin critique pour des réseaux capables de s'affranchir des contraintes physiques de la ville, notamment en termes de densité et de portée. Les solutions traditionnelles atteignent leurs limites face à des déploiements massifs, lorsqu'il s'agit de connecter des dizaines de milliers d'appareils, allant des compteurs électriques intelligents à l'éclairage public.

Pour répondre à ces problématiques, le standard Wi-SUN a été créé en janvier 2012. Il s'appuie sur une couche physique sub-GHz (norme IEEE 802.15.4g), offrant une meilleure propagation radio en milieu urbain dense et une grande résilience face aux obstacles physiques. Ce réseau maillé permet d'interconnecter des millions d'appareils grâce à l'adressage IPv6 et au protocole de routage RPL (Routing Protocol for Low-power and Lossy Networks). Au-delà de ses capacités d'auto-découverte et d'auto-réparation, il garantit la sécurité du réseau via le standard IEEE 802.1X, qui assure l'authentification de chaque nœud. Enfin, la Wi-SUN Alliance encadre l'interopérabilité entre les constructeurs, garantissant ainsi l'homogénéité et la pérennité des déploiements.

Zephyr est un RTOS en plein essor, soutenu par la Linux Foundation depuis sa première publication en février 2016 [1]. Il est conçu pour des appareils et des systèmes présentant de fortes contraintes mémoire. Son architecture repose sur un unique espace d'adressage ainsi qu'une gestion statique de la mémoire. Il reprend des concepts essentiels à Linux, tels que le DeviceTree et Kconfig, ce qui le rend hautement configurable et facilement adaptable à de nouvelles cibles matérielles. Néanmoins, à l'heure actuelle, il n'existe aucune implémentation open-source du standard Wi-SUN sur ce RTOS, malgré le fait qu'il intègre déjà nativement de nombreux protocoles réseau.

Nanostack est une pile de communication open-source intégrant le standard Wi-SUN. Actuellement, elle est étroitement liée à l'écosystème Mbed OS, un système qui n'est plus maintenu depuis 2024. Ce travail de Bachelor a pour but de pallier cette obsolescence en portant la Nanostack vers Zephyr et en l'intégrant pleinement à son architecture. À terme,

cette contribution permettra de déployer Zephyr dans un contexte de Smart Cities de manière entièrement open-source, tout en garantissant la pérennité de cette solution réseau.

2.1 Structure du document

Ce rapport intermédiaire s'articule autour des axes suivants :

- **Cahier des charges** : Définition des phases du projet et des livrables attendus.
- **Planification initiale** : Présentation du calendrier prévisionnel et des jalons du projet.
- **État de l'art** : Analyse des différentes technologies étudiées en vue du portage.
- **Stratégie de portage de la Nanostack** : Détail de la méthodologie employée pour adapter les différents composants, ainsi que l'argumentaire des choix techniques réalisés.
- **Bilan et planification actualisée** : État d'avancement des travaux au regard de la planification initiale et ajustement pour la suite du projet.

3 Cahier des charges

3.1 Résumé du contexte

L'essor des villes intelligentes (*Smart Cities*) rend indispensable le déploiement de réseaux capables d'interconnecter une multitude d'équipements. Ces applications imposent des contraintes réseau strictes : une densité élevée d'appareils, une couverture longue portée et une forte résilience face aux obstacles urbains.

Pour répondre aux défis de l'IoT (Internet des Objets), **Zephyr** s'est imposé comme une référence. Ce système d'exploitation temps réel (RTOS), lancé en 2016, est spécialement conçu pour les petits systèmes embarqués connectés et à faibles ressources. Il offre une connectivité avancée, de multiples API de communication et prend en charge un large éventail d'architectures 32 et 64 bits.

De son côté, la **Nanostack** est une pile de communication open source reconnue pour son intégration robuste des réseaux maillés, notamment via le protocole Wi-SUN. Cependant, elle a été historiquement développée pour le système d'exploitation Mbed OS, dont la maintenance officielle a pris fin au profit d'autres solutions.

3.1.1 Problématique

Mbed OS n'étant plus supporté, le projet consiste à porter la pile réseau Nanostack vers Zephyr RTOS afin de pérenniser son utilisation. L'enjeu technique principal réside dans l'adaptation de la couche d'abstraction matérielle (HAL) de la Nanostack pour qu'elle puisse interagir nativement avec l'écosystème et les pilotes matériels de Zephyr.

Ce portage s'articulera autour de deux phases majeures :

1. **Fonctionnement autonome (*Standalone*)** : Adapter et faire tourner le cœur de la Nanostack sur Zephyr de manière indépendante, en validant les interactions de bas niveau (radio, timers, gestion mémoire).

2. **Intégration système(Optional) :** Coupler intimement la Nanostack à la pile réseau native de Zephyr, afin qu'elle puisse être exploitée par les applications de haut niveau de manière totalement transparente, via l'API standard des *sockets* BSD.

3.2 Phase du projet

1. Prise en main de la base de code
 - Fork les sources de mbed-os
 - Fork les sources de zephyr-os
 - Prise en main de la hiérarchie des dossiers de la Nanostack
 - Mise en évidence des différents dossier et leur utilisation
2. Prise en main de Zephyr OS
 - Installation de la toolchain
 - Compiler le kernel ainsi qu'un exemple sur qemu
 - Compiler le kernel ainsi qu'un exemple sur une cible
 - Ecrire un petit programme de test
3. Mettre en évidence les différences de fonctionnement entre mbed-os et zephyr
 - Comprendre comment fonctionne la event-loop de mbed-os
 - Comprendre comment fonctionne les interfaces reseau de mbed-os
 - Comprendre comment fonctionne les driver réseau sur zephyr
 - Isoler les différents composants
4. Ecrire le rapport intermediaire
 - Mettre l'avancement du projet
 - Expliquer le fonctionnement de zephyr etc
5. Isoler la partie « métier » de la stack est isoler les fonctions lié au kernel
 - Copier la partie qui n'est pas relié a l'os
 - Commencer a faire l'inventaire des fonctions a porter
6. Ecrire la HAL(Hardware Abstraction Layer)
 - Remplacer toute les fonctions kernel appeler par la stack par des fonctions de la HAL
 - Implementer chacune de ses fonctions pour zephyr
7. Tester l'implémentation
 - Ecrire des test pour garantir le bon fonctionnement de la stack
 - Ecrire un petit programme zephyr qui va permettre de faire communiquer 2 boards entre elle avec la nanostack
8. Optional: Integrer la nanostack a l'écosystème reseau de Zephyr

- Porter la nanostack de façon à ce que on appelle les socket BSD pour que ça fonctionne

9. Rédaction du rapport final

- Documenter les implementations
- Documenter l'avancement du projet

3.2.1 Livrables

Les livrables seront les suivants :

- Le rapport complet
- Un repo Github contenant le portage de la Nanostack
- Un repo Github contenant un code d'exemple

4 Planification

4.1 Planification initiale

Phase	Tâches associées	Temps alloué	Proportion
1. Base de code	- Fork de mbed-os et zephyr-os - Analyse de la hiérarchie de la Nanostack	20 h	4%
2. Prise en main Zephyr	- Installation de la toolchain - Compilation kernel + exemples - Programme de test	40 h	9%
3. Étude comparative	- Analyse de l'évent-loop - Interfaces et drivers réseau - Isolation des composants	50 h	11%
4. Rapport intermédiaire	- Rédaction de l'état d'avancement - Explication théorique	20 h	4%

Phase	Tâches associées	Temps alloué	Proportion
5. Isolation métier/kernel	• Copie de la partie agnostique - Inventaire des fonctions à porter	50 h	11%
6. Création de la HAL	• Remplacement des appels kernel - Implémentation pour zephyr	120 h	27%
7. Phase de tests	• Écriture des tests de validation - Dev d'un programme de communication P2P	80 h	18%
8. Intégration (Optionnel)	• Portage pour appel via les sockets BSD	40 h	9%
9. Rapport final	• Documentation des implémentations - Bilan du projet	30 h	7%
Total estimé		450 h	100%

4.2 Diagramme de gantt initiale

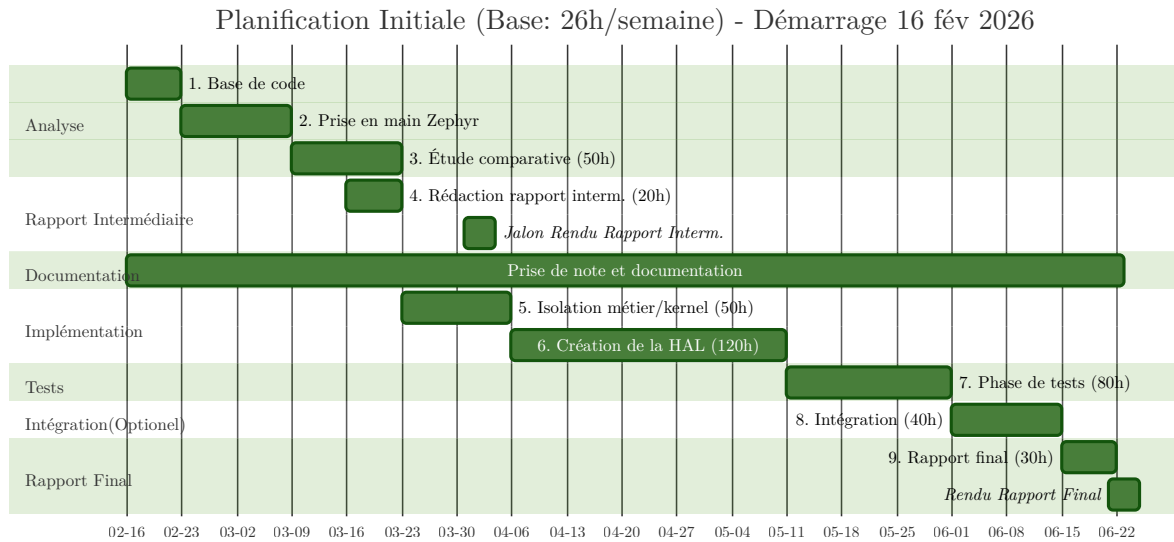


Fig. 1. – Planification Initiale

5 État de l'art

5.1 Standard Wi-SUN

Le standard Wi-SUN a été développé en janvier 2012 par la Wi-SUN alliance. Il répond à la montée des Smart Cities qui demandent de pouvoir connecter des centaines de milliers voir des millions de device ensemble dans un environnement urbain rempli d'obstacle physique. Pour parvenir à connecter tous ces appareils sans soucis de propagation d'onde et d'interférence, il utilise des normes étudiées dans ce but tel que le 802.15.4g. Pour le protocole de routage il utilise RPL (Routing Protocol for Low-Power and Lossy Networks) et un adressage IPv6. Il a été aussi conçu dans le but de minimiser la consommation d'énergie.

5.2 Mbed OS

Mbed OS est un système d'exploitation temps réel embarqué qui est basé sur CMSIS-RTOS qui est une API mise à disposition par ARM pour dialoguer plus simplement et uniformément peu importe le fabricant avec le processeur. Il intègre une mécanique temps réel ainsi que de gestion des threads.

Mbed OS intègre un grand nombre de module de connectivité et est particulièrement utilisé dans le monde de l'iot. Il implémente de nombreux protocoles de communication et met aux développeurs une implémentation haut niveau. Il supporte aussi un grand nombre de boards et a beaucoup de drivers.

Il a notamment la Nanostack, une pile de communication qui intègre entre autres le protocole Wi-SUN. Malheureusement ARM a décidé de déclarer la End Of Life de Mbed OS en juillet 2026. Ce qui a motivé l'intention de porter la Nanostack sur un nouvel RTOS pour bénéficier du standard Wi-SUN sur de nouveaux OS.

5.3 Zephyr

Zephyr est un système d'exploitation open source soutenu par la Linux Foundation, dont la version 1.0 a été publiée le 17 février 2016. Il est spécifiquement conçu pour les systèmes embarqués aux ressources limitées répondant à des contraintes temps réel. S'inspirant fortement de Linux, il en reprend des concepts standards tels que le Device Tree et Kconfig.

C'est un OS extrêmement modulaire qui prend en charge un vaste panel d'architectures matérielles. Pour simplifier le flux de travail, il s'appuie sur son propre outil en ligne de commande, west. Ce dernier permet de cross-compiler, de tester et de déployer aisément son code sur l'ensemble des plateformes compatibles.

5.3.1 Kernel Zephyr

Le noyau de Zephyr offre divers fonctionnalités pour permettre à l'utilisateur de développer son application sans soucis. Les services étant:

- Threading
- Ordonnancement
- Interruption
- Mécanisme de synchronisation
- Système d'événements
- Gestion de la mémoire

Il s'occupe aussi de la gestion des drivers. Le modèle de driver Zephyr ressemble au modèle Linux en ayant des drivers génériques par type d'interface qui doivent être écrits ensuite.

5.3.2 Gestion de l'énergie

5.3.3 Connectivité et protocole

5.4 Nanostack

La Nanostack est une pile de communication développée sur Mbed qui intègre plusieurs protocoles de communication. Comme Mbed OS est en fin de vie, il est nécessaire de porter la pile sur un nouvel OS.

Pour cela, il faut bien comprendre le fonctionnement de la Nanostack, son système d'événement.

La Nanostack est composée de 3 parties.

- Le SAL
- Le HAL

- L'event Loop

5.4.1 SAL (Socket Abstraction Layer)

La SAL gère toute la partie purement logicielle de la Nanostack, totalement indépendante du matériel (qui est délégué à la couche matérielle, la HAL). Son rôle principal est de masquer la complexité du réseau en offrant une interface de programmation standardisée (API Socket) à l'application.

C'est dans cette couche que s'exécutent les machines d'état des différents protocoles (MAC, 6LoWPAN, routage RPL, IPv6, TCP/UDP). Pour fonctionner efficacement sur des microcontrôleurs limités, la SAL s'appuie sur trois piliers :

- **Le Nanostack Event Loop** : Un ordonnanceur d'événements coopératif qui gère le trafic réseau de façon asynchrone pour économiser la RAM, sans nécessiter de multiples *threads*.
- **Une gestion interne du temps et de la mémoire** (`ns_timer` et `ns_dyn_mem`) : Un système de minuteurs virtuels pour les *timeouts* réseau et un allocateur de *buffers* optimisé pour prévenir la fragmentation de la mémoire lors d'un trafic intense.

Enfin, la SAL intègre nativement les mécanismes de sécurité (via Mbed TLS) pour chiffrer les communications avant leur transmission. Il est important de noter que bien que Mbed OS ait atteint sa fin de vie, le projet Mbed TLS demeure activement maintenu par la communauté TrustedFirmware [2]. Son intégration reste donc pertinente et pérenne pour ce projet, nous dispensant de chercher une solution cryptographique alternative.

Enjeux pour le portage : L'indépendance de la SAL vis-à-vis du matériel est un atout majeur pour la migration vers Zephyr. La logique complexe des protocoles (notamment Wi-SUN) n'a pas besoin d'être réécrite. Le défi consistera principalement à concevoir des adaptateurs pour lier l'Event Loop et les *timers* de la SAL aux primitives du noyau temps réel de Zephyr.

5.4.2 HAL (Hardware Abstraction Layer)

L'HAL s'occupe de faire le pont entre l'os hôte et la Nanostack. Il va s'occuper de relier les composants logiciels aux composants Hardware. Il doit aussi pouvoir s'occuper de communiquer avec les driver Ethernet et IEEE 802.15.4.

Ses domaines importants à implémenter sont :

- **Allocation de la mémoire** : Bien que la Nanostack a son propre système de mémoire dynamique, elle doit demander à son initialisation de la mémoire au kernel de son os hôte

- **Gestion du multithreading** : Etant donné que Mbed OS et Zephyr sont des RTOS

Bibliographie

- [1] « The Linux Foundation Announces Project to Build Real-Time Operating System for Internet of Things Devices | Zephyr Project ». Consulté le: 27 mars 2026. [En ligne]. Disponible sur: <https://web.archive.org/web/20160310073146/https://www.zephyrproject.org/news/linux-foundation-announces-project-build-real-time-operating-system-internet-things-devices>
- [2] « Important Update on Mbed - End of Life - Mbed OS ». Consulté le: 29 mars 2026. [En ligne]. Disponible sur: <https://forums.mbed.com/t/important-update-on-mbed-end-of-life/23644>

BIBLIOGRAPHIE

Figures

Fig. 1 Planification Initiale	17
-------------------------------------	----

FIGURES

Tables

Tableau 1 Glossaire des termes techniques	7
---	---